TEMA

Tema 8. Desarrollo de programas organizados en clases

Informática

Programación I

Autoría: Cristian Catalán

Linkia FP

Tema 8: Desarrollo de programas organizados en clases

¿Qué aprenderás?

- Definir getters y setters.
- Utilizar los distintos modificadores de acceso.
- Crear y utilizar clases heredadas.
- Crear y utilizar interfaces y clases abstractas.

¿Sabías que…?

- El concepto de “interfaz” se utiliza sobretodo en lenguajes orientados a objetos que no aceptan la herencia múltiple.
- El concepto de “abstract” difiere entre distintos lenguajes de programación.
- Algunas empresas como Tiboe se han especializado en calcular la calidad de un software.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

1. Encapsulamiento

Como ya vimos en el tema **Error! Reference source not found.** , los lenguajes orientados a objetos (como Java) tienen como una característica principal el "encapsulamiento". Consiste en poder limitar el acceso a de aquellas funciones que no sean estrictamente para utilizar la Clase.

Vamos a intentar entender la necesidad del encapsulamiento con el siguiente ejemplo. Hemos creado una clase Dni con un constructor que inicializa las variables y genera un password asociado al Dni. Ahora bien, ¿Cómo podría asegurar que el password solo se genera al crear el nuevo Dni?

Lo que tendremos que hacer es encapsular el atributo *password* y el método *generaPassword* para que solo puedan tener acceso a ellos la misma clase Dni. Para ello utilizaremos los modificadores de acceso y diremos que la *password* y *generaPassword* van a ser privados.

```
class Dni {
   String nombre;
   int edad;
   int numero;
   char letra;
   int password;

   public  Dni() {}

   public  Dni(int edad, int numero, char letra) {
      this.edad = edad;
      this.numero = numero;
      this.letra = letra;
      generaPassword();
      System.out.println("Password:"+password);
   }

/**Genera un password entre 100 y 500*/
   public void generaPassword() {
      password = (int) (Math.random() * 500 + 100);
   }
}
```

```
public class EjemploDni {
   public static void main(String[] args) {
      Dni objetoDniRyu;
      objetoDniRyu= new Dni(54,467374837,'W');
      objetoDniRyu.nombre="Ryu";
   }
}
```

Sin indicar nada desde *EjemploDni* se podría modificar el *password*. Lo evitaremos con los modificadores de acceso.

1.1. Modificadores de Acceso

Los modificadores de acceso nos indican qué clases van a poder utilizar cada método y atributo. Utilizando los modificadores de acceso vamos a poder determinar qué atributos y métodos van a ser usados solamente para la gestión interna de la clase y cuales para el propósito general de la clase.

Indicaremos los modificadores de acceso cuando definamos un método o un atributo con la siguiente sintaxis:

modificadorAcceso TipoVariable nombreVariable;

modificadorAcceso tipoRetorno nombreFuncion(){ .... }

Los posibles modificadores de acceso para atributos y métodos son:

- **private**: solo tienen acceso a estas variables y métodos la propia Clase.
- **Sin modificador**: solo tienen acceso a estas variables y métodos las clases definidas en el mismo package.
- **protected**: solo tienen acceso a estas variables y métodos las clases definidas en el mismo package y las Clases derivadas de la propia Clase.
- **public**: tienen acceso a estas variables y métodos todas las Clases que las quieran usar.

Vamos a modificar el código de ejemplo encapsulándolo utilizando el modificador *private* para encapsular *password* y *generaPassword*().

```
class Dni {
   String nombre;
   int edad;
   int numero;
   char letra;
private   int password;

   public  Dni() {}

   public  Dni(int edad, int numero, char letra) {
      this.edad = edad;
      this.numero = numero;
      this.letra = letra;
      generaPassword();
      System.out.println("Password:"+password);
   }

/**Genera un password entre 100 y 500*/
   private void generaPassword() {
      password = (int) (Math.random() * 500 + 100);
   }
}
```

```
public class EjemploDni {
   public static void main(String[] args) {
      Dni objetoDniRyu;
      objetoDniRyu= new Dni(54,467374837,'W');
      objetoDniRyu.nombre="Ryu";
   }
}
```

private → Solo la clase
protected → clase + hijas
public → Todo el mundo

IMPLE ALUMNO - LINKIAFP

Viendo el ejemplo anterior, nos preguntamos qué modificadores de acceso debemos utilizar para el resto de atributos de Dni. ¿Es correcto que cualquier programa que utilice mi clase Dni vaya a poder acceder directamente a los atributos que quiera en un objeto Dni? Si así lo fuera los tendría que definir con el modificador *public*.

¿Pero lógicamente un Dni va a poder contener cualquier valor como nombre, edad, número y letra? Seguramente no (una edad negativa o un nombre vacío no parece algo con mucho sentido). Deberemos entonces aplicar los modificadores *private* a los atributos y utilizar algún mecanismo para poder modificar y/o leer ésos valores.

Como convenio utilizaremos las funciones *Setters* y *Getters*.

2. Setters y Getters

En la mayoría de ocasiones los atributos de una Clase no van a poder ser modificados con cualquier valor y en otras ocasiones deberemos hacer alguna operación antes de retornar el atributo. Para gestionar estas operaciones de asignación y obtención de atributos utilizaremos las funciones que clasificaremos como Setters y Getters.

2.1. Setters

Setter es una nomenclatura utilizada para indicar que el objetivo de un método es establecer (en inglés Set) el valor de un atributo o varios.

Por convenio el nombre de una función Setter va a empezar por la palabra ***set*** seguido del nombre del atributo a modificar.

Las funciones Setters normalmente deberían tener un modificador de acceso ***public*** para que cualquier clase pudiera utilizarlas para establecer los valores al objeto.

En general una función Setter va a recibir como parámetros como mínimo un valor (el valor a establecer) y si retorna algún valor será un indicador de si se ha podido o no establecer el valor indicado al atributo.

En el siguiente código de ejemplo podemos ver como todos los atributos están privados y para establecer sus valores se han definido una serie de Setters. Aprovechando el encapsulamiento hemos forzado a que el número y letra del DNI se tenga que indicar conjuntamente para así poder comprobar si los valores se corresponden o hay algún error. También controlamos que la edad sea un valor entre 0 y 150. Para simplificar el código hemos dejado solo el constructor por defecto.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

```
class Dni {
   private String nombre;
   private int edad;
   private int numero;
   private char letra;
   private int password;

public     Dni() {
      generaPassword();
      System.out.println("Password:" + password);
   }
   private void generaPassword() {
      password = (int) (Math.random() * 500 + 100);
   }

   public void setNombre(String nombre) {
      this.nombre = nombre;
   }

   public boolean setEdad(int edad) {
      if (edad > 0 && edad < 150) {
         this.edad = edad;
         return true;
      }
      return false;
   }

   public boolean setNumeroDni(int numero, char letra) {
      if (checkLetra(numero, letra)) {     return true; }
      return false;
   }

   public boolean checkLetra(int numero, int letra) {
      if (letra == "TRWAGMYFPDXBNJZSQVHLCKE"
                        .toCharArray()[numero % 23]) {   return true;
      }
      return false;
   }
}
```

```
public class EjemploDni {

   public static void main(String[] args) {
      Dni objetoDniRyu = new Dni();
      objetoDniRyu.setNombre("Ryu");
      boolean edadOk = objetoDniRyu.setEdad(54);

      boolean dniOk = objetoDniRyu
                        .setNumeroDni(467374837, 'W');

      System.out.println("Edad correcta? " + edadOk);
      System.out.println("DNI correcto? " + dniOk);

   }
}
```

Ejemplo de Setters para controlar la asignación de valores a atributos de una clase

Ahora vamos a ver cómo podemos controlar el acceso a los atributos de una clase para obtener sus valores.

2.2. Getters

Getter es una nomenclatura utilizada para indicar que el objetivo de un método es obtener (en inglés Get) el valor de un atributo o un valor derivado.

Por convenio el nombre de una función Getter va a empezar por la palabra ***get*** seguido del nombre del atributo a modificar.

Las funciones Getters normalmente deberían tener un modificador de acceso ***public*** para que cualquier clase pudiera utilizarlas para obtener los valores del objeto.

Una función Getter debería retornar el valor del atributo que se quiere obtener y recibirá como parámetros aquellos valores necesarios para la obtención del valor. Las funciones Getters no tienen por qué limitarse solo a los atributos definidos en una Clase, también se pueden utilizar para obtener valores derivados y calculados de los atributos de una clase.

Por ejemplo, en el código que veremos a continuación no vamos a definir exactamente un Getter para cada atributo, si definiremos un Getter para *nombre* y otro para *edad*, pero también uno que nos retorne una String con el número y letra del DNI conjuntamente y otro que nos indique si el Dni pertenece a una persona mayor de edad o no.

Para simplificar el ejemplo hemos eliminado los setters y creado un constructor con todos los atributos.

```
class Dni {
   private String nombre;
   private int edad;
   private int numero;
   private char letra;
   private int password;
   public  Dni() {
      generaPassword();
      System.out.println("Password:" + password);
   }
   private void generaPassword() {
      password = (int) (Math.random() * 500 + 100);
   }
   public String getNombre(){
      return nombre;
   }
   public int getEdad(){
      return edad;
   }
   public String getDni(){
      return numero+""+letra;
   }
   public boolean getEsMayorEdad() {
      return edad >= 18;

}
```

```
public class EjemploDni {
   public static void main(String[] args) {
      Dni objetoDniRyu;
      objetoDniRyu= new Dni(54,467374837,'W',"Ryu");
      String nombre=objetoDniRyu.getNombre();

      int edad=objetoDniRyu.getEdad();
      String dni=objetoDniRyu.getDni();
      boolean mayor=objetoDniRyu.getEsMayorEdad();

      System.out.println("Nombre:"+nombre);
      System.out.println("Edad:"+edad);
      System.out.println("DNI:"+dni);
      System.out.println("Mayor edad?:"+mayor);
   }
}
```

3. Herencia en la POO

Cuando empecemos a crear clases para estructurar la información nos vamos a encontrar en múltiples ocasiones con Clases muy parecidas, pero con algún método o atributo adicional. Gracias a la herencia podremos evitar el código duplicado indicando que una clase va heredar de otra.

La Herencia es una de las características principales de cualquier lenguaje de programación orientado a objetos. Es el mecanismo por el cual una clase adquiere los atributos y métodos de otra clase. Nos referiremos a la clase de la que se hereda como *superclase* o *clase madre* y la clase heredada como clase hija o clase *extendida* o *derivada.*

En la POO encontraremos dos tipos distintos de herencia:

- Herencia simple, en la que una clase va a poder heredar los métodos y atributos de una única clase. Como es el caso de Java o C#.
- Herencia múltiple, en la que na clase va a poder heredar de múltiples clases. Como es el caso de C++.

Vamos a ver la necesidad de la herencia con el siguiente ejemplo compuesto por la clase *Alimento* y la clase *AlimentoSolido*. Ambas clases comparten las características de *Alimento*, pero *AlimentoSolido* a demás tiene el atributo dureza y el método masticar:

```
public class Alimento {
   String nombre;
   int cantidad;
   int kcal;
  public  Alimento(String nombre, int cantidad, int kcal) {
      this.nombre=nombre;
      this.cantidad=cantidad;
      this.kcal= kcal;
   }
}
```

```
public class AlimentoSolido {
   String nombre;
   int cantidad;
   int kcal;
   int dureza;
public  AlimentoSolido (String bistec, int cantidad,
                                int kcal,int dureza) {
      this.nombre=nombre;
      this.cantidad=cantidad;
      this.kcal= kcal;
      this.dureza=dureza;
   }
   void masticar(int intensidad){
       dureza-=intensidad;
       if(dureza<0) dureza=0;
   }
}
```

La clase AlimentoSolido es clara candidata a heredar de Alimento

Como buenos programadores que somos, la idea de tener código duplicado no nos gusta nada. ¿Podríamos conseguir que la clase *AlimentoSolido* contenga todos los atributos y métodos de la clase *Alimento*? La respuesta por suerte es que si, solo tendremos que indicar que la clase *AlimentoSolido* va a heredar todos los atributos y métodos de la clase *Alimento* con la palabra clave **extends**.

Lo único que no vamos a heredar van a ser los constructores con la excepción del constructor vacío. Por eso va a ser obligatorio que todas las superclases tengan definidas un constructor vacío (sin parámetros).

La sintaxis que utilizaremos es:

class NombreClaseHija extends NombreClaseMadre{}

Si utilizamos extends en nuestro ejemplo anterior obtendremos las siguientes clases:

```
public class Alimento {
   String nombre;
   int cantidad;
   int kcal;
   Alimento(){ }
   Alimento(String nombre, int cantidad, int kcal) {
      this.nombre=nombre;
      this.cantidad=cantidad;
      this.kcal= kcal;
   }
}
```

```
public class AlimentoSolido extends Alimento {
   int dureza;
   public AlimentoSolido(String bistec, int cantidad,
                                 int kcal, int dureza) {
      this.nombre = nombre;
      this.cantidad = cantidad;
      this.kcal = kcal;
      this.dureza = dureza;
   }

   void masticar(int intensidad) {
      dureza -= intensidad;
      if (dureza < 0) {
         dureza = 0;
      }
   }
}
```

La clase AlimentoSolido hereda de Alimento

Como podemos observar en el ejemplo anterior, los atributos y funciones definidas en la clase madre automáticamente se heredan en la clase hija. Podemos acceder sin problemas al nombre, cantidad o kcal como si fueran atributos definidos en la propia clase.

Pero vemos que tenemos un buen trozo del código del constructor de AlimentoSolido repetido en el constructor de Alimento. ¿Cómo podríamos llamar al constructor de la superclase Alimento en el constructor de AlimentoSolido? Para conseguirlo utilizaremos la palabra clave **super**.

VERSIÓN IMPRIMIBLE INKIAFP

3.1. El uso de super y constructor

Podemos pensar que la palabra clave *super* es el *this* de la superclase. Así cuando escriba *super* obtendré acceso a todos los métodos y atributos de la superclase, y si estoy dentro de un constructor podré acceder directamente al constructor de la superclase.

Para acceder al constructor de la superclase utilizaremos la sintaxis:

super(parámetros del constructor);

Para acceder a un método definido en la superclase utilizaré la sintaxis:

super.nombreMetodo();

Para acceder a un atributo definido la superclase utilizaré la sintaxis:

super.nombreAtributo;

Siguiendo con el mismo ejemplo vamos a utilizar **super** para llamar desde *AlimentoSolido* al constructor de la clase *Alimento* que establece el nombre, cantidad y kcal de un alimento. A demás añadiremos a AlimentoSolido un constructor con los mismos parámetros que el constructor de Alimento.

```
public class Alimento {
   String nombre;
   int cantidad;
   int kcal;
   Alimento(){ }
   Alimento(String nombre, int cantidad, int kcal) {
      this.nombre=nombre;
      this.cantidad=cantidad;
      this.kcal= kcal;
   }
}
```

```
public class AlimentoSolido extends Alimento {
   int dureza;
//constructor con el parametron dureza
   public AlimentoSolido(String bistec, int cantidad,
                         int kcal, int dureza) {
      super(bistec,cantidad,kcal);
      this.dureza = dureza;
   }
//constructor con los mismos parámetros que Alimento
   public AlimentoSolido(String bistec, int cantidad,
                         int kcal){
      super(bistec,cantidad,kcal);
   }
   void masticar(int intensidad) {
      dureza -= intensidad;
      if (dureza < 0) {
         dureza = 0;
      }
   }
}
```

La clase AlimentoSolido hereda de Alimento y utiliza **super** para llamar a un constructor de la superclase

VERSIÓN ALUMNOS LINKIAFP

En el ejemplo anterior se ve claramente el uso de **super** para reutilizar un constructor de la superclase, pero ¿para qué vamos a querer llamar a un método o acceder a un atributo de la superclase mediante **super** si sus valores se han heredado? Es decir, si podemos acceder directamente a los atributos y métodos heredados de la superclase, ¿para qué nos sirve acceder mediante **super**?

Un pequeño atisbo de la respuesta lo observamos cuando en la clase hija hemos definido un constructor con los mismos parámetros que el constructor de la superclase sobrescribiendo el constructor de la superclase. Y es que en general vamos a poder sobrescribir los atributos y métodos de la superclase.

3.2. Sobrescribiendo métodos

Cuando una clase hereda de otra, la clase hija va a poder sobrescribir los métodos y atributos de la superclase siempre que no se haya definido con la palabra clave **final**. Solo podemos sobrescribir el método con los mismos tipos y parámetros que en la superclase o los atributos a los que queramos modificar sus valores.

Si sobrescribimos un atributo asignándole un nuevo valor, podemos utilizar la palabra clave **super** para acceder al valor asignado por la superclase.

Cuando sobrescribimos un método podemos utilizar la palabra clave **super** para acceder a la implementación del método en la superclase.

En el siguiente ejemplo hemos añadido a *Alimento* un atributo *unidadMedida* y una función *toString()* que hemos sobrescrito en *AlimentoSolido*. Para mostrar cómo actúa la palabra clave **super,** en el constructor de *AlimentoSolido* se muestra el valor de *unidadMedida* con y sin **super** y ejecutamos la función *toString()* con y sin **super**.

```
public class EjemploHerencia {
   public static void main(String[] args) {
      AlimentoSolido bistec;
      bistec = new AlimentoSolido("bistec",100,400,20);
   }
}
```

```
public class Alimento {
   String nombre;
   int cantidad;
   String unidadMedida=" ";
   int kcal=0;
   Alimento(){ }
   Alimento(String nombre, int cantidad, int kcal) {
      this.nombre=nombre;
      this.cantidad=cantidad;
      this.kcal= kcal;
   }
   public String toString(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida;
   }
}
```

```
public class AlimentoSolido extends Alimento {
   int dureza;
   String unidadMedida="gramos";
   public AlimentoSolido(String bistec, int cantidad
                                          , int kcal, int dureza) {
      super(bistec,cantidad,kcal);
      this.dureza = dureza;

      System.out.println("medida:"+super.unidadMedida);
      System.out.println("medida:"+unidadMedida);

      System.out.println(super.toString());
      System.out.println(toString());
   }
   public String toString(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida+
            ",dureza:"+dureza;
   }
}
```

Tras ejecutar el código anterior se mostrará por consola los siguientes valores:

medida:

medida:gramos

nombre:bistec,cantidad:100

nombre:bistec,cantidad:100gramos,dureza:20

Vemos entonces como el primer valor de *unidadMedida* que se muestra es una cadena vacía que se corresponde con el valor almacenado en la superclase Alimento. El segundo valor de *unidadMedida* que se muestra es "gramos", que se corresponde con el indicado en la clase *Alimento*. Igualmente el valor que retorna *super.toString()* se corresponde con la implementación de la función realizada en la superclase *Alimento*. Y en cambio el valor que retorna *toString()* se corresponde con la implementación de la subclase *AlimentoSolido*.

Ahora nos podríamos preguntar si siempre va a ser posible extender de un atributo o un método, la respuesta sería que no. Podemos indicar que el valor de un atributo es inalterable o que no se puede sobrescribir un método añadiendo el modificador **final** en la definición del método o el atributo.

Por ejemplo, si queremos definir una variable *PI* con un valor inalterable y una función *calculaLongitudCircunferencia* que no se pueda sobrescribir los declararemos de la siguiente manera:

```
final float PI = 3.1415926535897932;
final  public float calculaLongitudCircunferencia(float radio){
        return 2*PI*radio;
}
```

Vídeo: Herencia_sobreescritura.wmv :Herencias y sobre escritura de métodos

En el último ejemplo hemos utilizado la clase *AlimentoSolido* sobrecargar el método *toString*() definido en Alimento, pero la realidad es que la clase *Alimento* ya estaba sobrecargando *toString*(), ¿de quién? De la superclase de la que heredan todas las clases que creemos, la clase *Object*.

4. Objeto Object

Object es una clase definida en package java.lang; de la que todas las nuevas clases van a heredar de forma automática todos sus métodos. Los principales métodos que vamos a heredar son:

String toString()	Retorna un texto que representa el objeto. Recomendable sobrescribirlo para que muestre un mensaje más agradable.
Object clone()	Retorna un nuevo objeto con la misma estructura y valores que el propio objeto que ha ejecutado la función. Se debe sobrescribir e implementarlo.
Boolean equals(Object obj)	Indica si el objeto actual es igual que el parámetro.
void finalize()	Lo llama automáticamente el garbage collector para borrar definitivamente un objeto. No lo llamaremos nosotros manualmente.
int hashCode ()	Retorna un código hash utilizado en las colecciones.

Pensando en la herencia y en como todas las clases que comparten una superclase comparten también sus métodos y atributos aparece el concepto de Polimorfismo, en donde en una superclase vamos a poder almacenar todos los datos compartidos de una subclase. Vamos a ver este concepto con más profundidad a continuación.

VERSIÓN DISPONIBLE ALUMNOS LINKS FP

5. Wrapper

Los Wrappers son las clases que representan variables primitivas. Los wrappers ofrecen un conjunto de métodos y atributos para trabajar con un tipo primitivo.

A continuación se muestran los tipos primitivos y sus wrappers:

Tipo primitivo	Wrapper asociado
byte	Byte
short	Short
int	Int
long	Long
float	Float
double	Double
character	Character
boolean	Boolean

Podemos crear una variable a partir de un wrapper igual que con cualquier otro tipo de clase. Por ejemplo, para crear una variable que sea un wrapper de int:

```
Integer numero = new Integer("89");
Integer numero = new Integer(5);
```

A demás también podemos crear una variable de un wrapper asignando a la variable directamente la variable primitiva que represente el wrapper.

```
Integer numero =55;
Boolean resul = false;
```

¿para que sirve?

VERSIÓN IMPRIMIBLE ALUMNO LANKIAN

Vamos a ver algunos de los principales métodos del wrapper Integer:

Método	Descripción
new Integer(int)	Constructor que recibe un valor entero
new Integer(String)	Constructor que recibe un valor String
Integer.parseInt(String)	Convierte una string a un valor entero
.byteValue() .shortValue() .intValue() .longValue() .doubleValue() .floatValue()	Convierte el valor del wrapper a una variable primitiva

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Recursos y enlaces

- Object, documentación oficial
  https://docs.oracle.com/javase/10/docs/api/java/lang/Object.html
- Documentación oficial sobre los modificadores disponibles en Java 10:
  https://docs.oracle.com/javase/10/docs/api/javax/lang/model/element/Modifier.html

Conceptos clave

- **superclase**: clase de la que hereda otra clase.
- **clase padre**: es la superclase.
- **super**: hace referencia a la superclase.
- **extends**: palabra clave para indicar que una clase hereda de otra.
- **wrapper:** se dice de aquellas clases que representan variables primitivas.

VERSIÓN IMPRIMIBLE ALUMNO EUKIAFP

Test de autoevaluación

¿Para qué nos sirven los modificadores de acceso?

a) Para implementar el polimorfismo.

b) Para implementar la abstracción.

c) Para implementar el encapsulamiento.

d) Para implementar la sobrecarga.

¿Qué modificador indica que no se puede sobrescribir un método?

a) final

b) static

c) global

d) constant

Ponlo en práctica

Actividad 1

Vamos a seguir con la actividad del tema anterior que dice:

"Queremos estructurar la información de los habitantes de una ciudad extraterrestre formada por dos razas distintas: los Cabecines y los Cabezones. De cada individuo guardaremos su nombre y su color de piel (los individuos de ambas razas pueden tener nombres y colores distintos al resto). A demás queremos guardar para cada raza su nombre y el número de espacios que ocupan teniendo en cuenta que los Cabecines ocupan 1 espacio y los Cabezones ocupan 2.

Ambas razas van a tener una función de nombre *toString()* que retorne una string con los atributos del individuo.

Ambas razas van a tener una función de nombre *estresar()* que cuando se ejecute cambiaré el color de piel de los Cabezones a azul y el color de piel de los Cabecines a rojo.
"

Añade ahora a los Cabezones un tercer ojo que puede ser "marrón" , "negro" o "rojo".

Crea la superclase Extraterrestre de la que hereden Cabecin y Cabezon. La clase Extraterrestre tendrá el atributo *raza* con valor "extraterrestre", el atributo *espacio* con valor inicial de 1.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Crea ahora una clase principal de nombre “CabezaWordl” que al ejecutarse cree 2 Cabecines y 2 Cabezones estableciendo su nombre y color con un constructor.

En el caso de los Cabezones establece también el color de su tercer ojo mediante un setter que establezca el tercer ojo como “rojo” si no se introduce un valor correcto.

Estresa a uno de cada y muestra sus valores por consola utilizando la función “toString”.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

SOLUCIONARIOS

Test de autoevaluación

¿Para qué nos sirven los modificadores de acceso?

a) Para implementar el polimorfismo.

b) Para implementar la abstracción.

c) Para implementar el encapsulamiento.

d) Para implementar la sobrecarga.

¿Qué modificador indica que no se puede sobrescribir un método?

e) final

f) static

g) global

h) constant

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Ponlo en práctica

Actividad 1

Vamos a seguir con la actividad del tema anterior que dice:

"Queremos estructurar la información de los habitantes de una ciudad extraterrestre formada por dos razas distintas: los Cabecines y los Cabezones. De cada individuo guardaremos su nombre y su color de piel (los individuos de ambas razas pueden tener nombres y colores distintos al resto). A demás queremos guardar para cada raza su nombre y el número de espacios que ocupan teniendo en cuenta que los Cabecines ocupan 1 espacio y los Cabezones ocupan 2.

Ambas razas van a tener una función de nombre *toString()* que retorne una string con los atributos del individuo.

Ambas razas van a tener una función de nombre *estresar()* que cuando se ejecute cambiaré el color de piel de los Cabezones a azul y el color de piel de los Cabecines a rojo.

Añade ahora a los Cabezones un tercer ojo que puede ser "marrón" , "negro" o "rojo".

Crea la superclase Extraterrestre de la que hereden Cabecin y Cabezon. La clase Extraterrestre tendrá el atributo *raza* con valor "extraterrestre", el atributo *espacio* con valor inicial de 1.

Crea ahora una clase principal de nombre "CabezaWordl" que al ejecutarse cree 2 Cabecines y 2 Cabezones estableciendo su nombre y color con un constructor.

En el caso de los Cabezones establece también el color de su tercer ojo mediante un setter que establezca el tercer ojo como "rojo" si no se introduce un valor correcto.

Estresa a uno de cada y muestra sus valores por consola utilizando la función "toString".

El solucionario está disponible en la versión interactiva del aula.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAD